Information Technology University




# Manual

*for*

# CS220: Artificial Intelligence

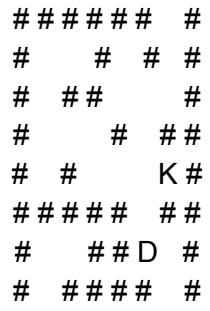
| |
|---|
| Course Instructor : **Husnain Haider** |
| Semester Fall 2023 : **3** |
| Teaching Assistant : **Muhammad Ahmad** |
| Teaching Assistant : **Moeed Ahmad** |
| Teaching Assistant : **Muzamil Ali** |
| Teaching Assistant : **Abdul Rehman Ameen** |
| Teaching Assistant : **Muhammad Anas** |

| |
|---|
| Student Name : |
| Student Roll No. : |

Department of Computer Science ITU, Lahore, Pakistan

## Lab-07

# Lab Manual: Labyrinth Search and Traversal




```
labyrinth = [
    [0, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 0, 1, 1, 1, 0],
    [0, 1, 1, 1, 1, 0, 0, 0],
    [0, 1, 0, 1, 1, 1, 3, 0],
    [0, 0, 0, 0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0, 2, 1, 0],
    [0, 1, 0, 0, 0, 0, 1, 0]
]
```

### Task 1: Drawing the Labyrinth

**Objective:**

Write a function that takes a labyrinth matrix as input and visualizes it using specific characters:

- `#` for walls
- Two spaces for empty passages
- `D` for doors
- `K` for keys

Additionally, the function should accept a list of `(x, y)` tuples representing positions visited by a robot. At these positions, print the character `.` along with either whitespace, `D`, or `K` if applicable (but not `#`, since it's reserved for walls).

**Example:**

def draw_labyrinth(labyrinth, visited):

**Input:**

- A 2D matrix representing the labyrinth where `1` is a passage, `0` is a wall, `D` represents a door, and `K` represents a key.

- A list of tuples `(x, y)` representing positions visited by the robot.

**Output:**

- A visual representation of the labyrinth.

## Task 2: Finding Adjacent Passages

**Objective:**

Write a function that, given the labyrinth matrix and specific `x, y` coordinates, returns a list of adjacent coordinates (tuples) that are valid passages (i.e., cells with value `1`). Ensure the function handles boundary cases properly and doesn't access memory outside the labyrinth array.

**Example:**

def get_adjacent_passages(labyrinth, x, y):

**Input:**

- A 2D matrix representing the labyrinth.
- Coordinates `x` and `y`.

**Output:**

- A list of adjacent coordinates that are passages.

## Task 3: Depth-First Search in the Labyrinth

**Objective:**

Implement a function (recursive or non-recursive) that finds a path from the entrance `(0, 6)` to the exit `(7, 1)` in a large labyrinth. The function should perform depth-first search (DFS). It must return the path once the goal is reached. You should keep track of the path taken and ensure that previously visited states are not revisited.

**Additional Condition:**

- The robot must collect key `K` before accessing door `D`.

**Hint:**

- First find a path till a key is found, then find the exit.

**Example:**

def find_path(labyrinth, start, end, path=[]):

**Input:**

- A 2D labyrinth matrix.
- Start coordinates `(0, 6)`.
- End coordinates `(7, 6)`.

**Output:**

- A list of coordinates representing the path from start to exit.

**Approaches:**

There are two approaches to this:

- One mentioned above
- Backtracking **(Bonus)**

Backtracking approach is a bonus task, if you are able to do it you will be given bonus marks, if not, there will be no deduction.

**Main (code):**

```
labyrinth = [
    [0, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 0, 1, 1, 1, 0],
    [0, 1, 1, 1, 1, 0, 0, 0],
    [0, 1, 0, 1, 1, 1, 3, 0],
    [0, 0, 0, 0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0, 2, 1, 0],
    [0, 1, 0, 0, 0, 0, 1, 0]
]

start = (0, 6)
end = (7, 6)
path = find_key_door_exit(labyrinth, start, end)
print("Complete path:", path)
```

**Notes:**

- Ensure that your solution handles edge cases such as boundaries and obstacles in the labyrinth.
- Properly document for clarity.

**Deliverables:**

- Submit the ipynb / py file.